

NUMPY & PANDAS DATA ANALYTICS ASSIGNMENT

TASK-1: NumPy Introduction & Arrays

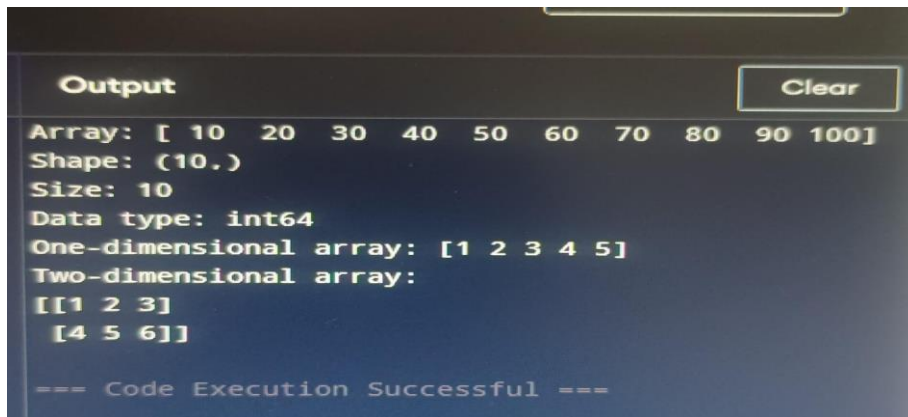
Program

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

print("Array:", arr)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data type:", arr.dtype)
one_d = np.array([1, 2, 3, 4, 5])
two_d = np.array([[1, 2, 3], [4, 5, 6]])
print("One-dimensional array:", one_d)
print("Two-dimensional array:")
print(two_d)
```

Output:

A screenshot of a Jupyter Notebook output cell. The output is displayed in a dark-themed interface. At the top, there is a tab labeled 'Output' and a 'Clear' button. The output text is as follows:

```
Array: [ 10  20  30  40  50  60  70  80  90 100]
Shape: (10,)
Size: 10
Data type: int64
One-dimensional array: [1 2 3 4 5]
Two-dimensional array:
[[1 2 3]
 [4 5 6]]
=== Code Execution Successful ===
```

Explanation:

- NumPy is a Python library used for working with numerical data and arrays.
- The program creates 1D and 2D arrays using `np.array()`.
- It displays the array's shape, size, and data type using `shape`, `size`, and `dtype`.
- A 1D array stores elements in one row, while a 2D array stores elements in rows and columns.

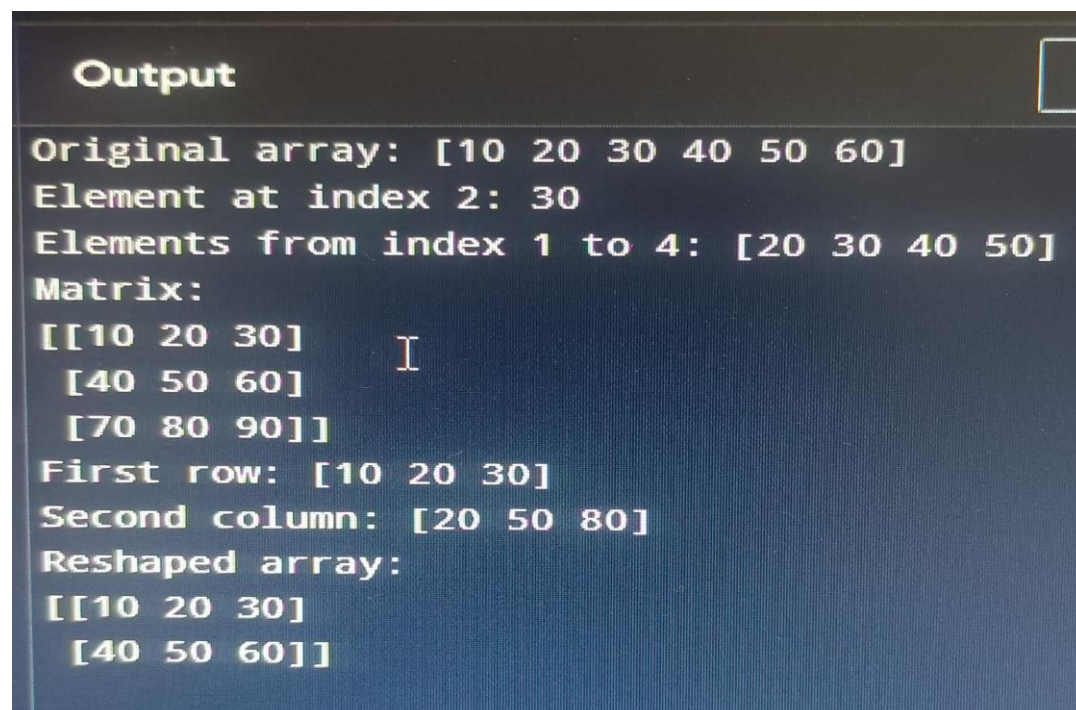
Task-2: NumPy Indexing, Slicing & Reshaping

Program

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60])
print("Original array:", arr)
print("Element at index 2:", arr[2])
print("Elements from index 1 to 4:", arr[1:5])
matrix = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])

print("Matrix:")
print(matrix)
print("First row:", matrix[0])
print("Second column:", matrix[:, 1])
reshaped = arr.reshape(2, 3)
print("Reshaped array:")
print(reshaped)
```

Output:



```
Output
Original array: [10 20 30 40 50 60]
Element at index 2: 30
Elements from index 1 to 4: [20 30 40 50]
Matrix:
[[10 20 30]
 [40 50 60]
 [70 80 90]]
First row: [10 20 30]
Second column: [20 50 80]
Reshaped array:
[[10 20 30]
 [40 50 60]]
```

Explanation:

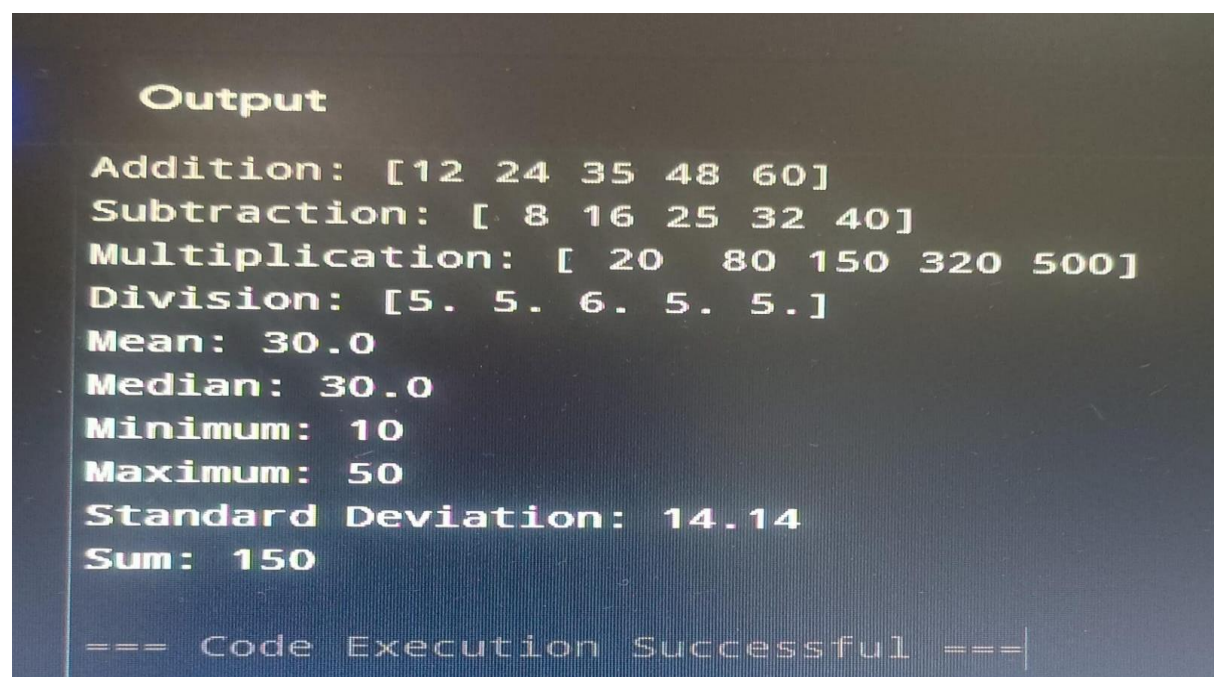
- NumPy indexing and slicing are used to access specific elements or ranges of elements in an array.
- The program accesses an element using `arr[2]` and a range using `arr[1:5]`.
- For the matrix, `matrix[0]` gets the first row and `matrix[:,1]` gets the second column.
- `reshape(2,3)` changes the 1D array into a 2×3 two-dimensional array without changing its elements.

Task-3: NumPy Mathematical & Statistical Operations

Program

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
b = np.array([2, 4, 5, 8, 10])
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
print("Standard Deviation:", round(np.std(a), 2))
print("Sum:", np.sum(a))
```

Output:



```
Output
Addition: [12 24 35 48 60]
Subtraction: [ 8 16 25 32 40]
Multiplication: [ 20  80 150 320 500]
Division: [5.  5.  6.  5.  5.]
Mean: 30.0
Median: 30.0
Minimum: 10
Maximum: 50
Standard Deviation: 14.14
Sum: 150

=== Code Execution Successful ===
```

Explanation:

- NumPy arithmetic and statistical operations are used to perform calculations on arrays quickly and efficiently.
- The program performs addition, subtraction, multiplication, and division between arrays a and b.
- It calculates statistical values such as mean, median, minimum, maximum, standard deviation, and sum of array a.
- Functions like np.mean(), np.median(), np.min(), np.max(), np.std(), and np.sum() are used for these calculations.

Task-4: Pandas Series & DataFrame**Program**

```
import pandas as pd
marks = pd.Series([85, 78, 92, 88, 76],
                  index=["Asha", "Ravi", "John", "Priya", "Kiran"])
df = pd.DataFrame({
    "Name": ["Asha", "Ravi", "John", "Priya", "Kiran"],
    "Age": [20, 21, 20, 22, 21],
    "Marks": [85, 78, 92, 88, 76]
})
print("Pandas Series:")
print(marks)
print("\nOriginal DataFrame:")
print(df)
print("\nColumn names:", list(df.columns))
print("Index:", list(df.index))
df["Result"] = ["Pass", "Pass", "Pass", "Pass", "Pass"]
print("\nUpdated DataFrame:")
print(df)
```

Output:

```
Output

Pandas Series:
Asha      85
Ravi      78
John      92
Priya     88
Kiran     76
dtype: int64

Original DataFrame:
   Name  Age  Marks
0  Asha   20    85
1  Ravi   21    78
2  John   20    92
3  Priya  22    88
4  Kiran  21    76

Column names: ['Name', 'Age', 'Marks']
Index: [0, 1, 2, 3, 4]

Updated DataFrame:
   Name  Age  Marks  Result
0  Asha   20    85    Pass
1  Ravi   21    78    Pass
2  John   20    92    Pass
3  Priya  22    88    Pass
4  Kiran  21    76    Pass

=== Code Execution Successful ===
```

Explanation:

1. Pandas Series and DataFrame are used to store and manage structured data in Python.
2. The program creates a Series to store students' marks with their names as indexes.
3. It creates a DataFrame containing students' names, ages, and marks, then displays the column names and index.
4. A new Result column is added to the DataFrame, showing the result of each student as "Pass".

Task-5: Reading & Inspecting Data

Program

```
import pandas as pd
df = pd.read_csv("sales_dataset.csv")
print("First 5 rows:")
print(df.head())
print("\nLast 5 rows:")
print(df.tail())
print("\nRows and columns:", df.shape)
print("\nColumn names:", list(df.columns))
print("\nData types:")
```



```

print(df.dtypes)
print("\nDataset information:")
print(df.info( ))
print("\nDescriptive statistics:")
print(df.describe())

```

Output:

```

*** First 5 rows:
  Order_ID  Product  Category Region  Quantity  Price  Sales
0      101    Laptop  Electronics  South      2   50000  100000
1      102     Phone  Electronics  North      3   25000   75000
2      103  Headphones  Accessories  South      5    2000   10000
3      104     Tablet  Electronics  East      2   30000   60000
4      105     Laptop  Electronics  West      1   55000   55000

Last 5 rows:
  Order_ID  Product  Category Region  Quantity  Price  Sales
5      106     Phone  Electronics  North      4   24000   96000
6      107     Mouse  Accessories  South      6    1200    7200
7      108  Keyboard  Accessories  West      3    1800    5400
8      109     Tablet  Electronics  East      2   32000   64000
9      110     Phone  Electronics  South      5   26000  130000

Rows and columns: (10, 7)

Column names: ['Order_ID', 'Product', 'Category', 'Region', 'Quantity', 'Price', 'Sales']

Data types:
Order_ID    int64
Product     str
Category     str
Region      str
Quantity    int64
Price       int64
Sales       int64
dtype: object

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Order_ID    10 non-null    int64
1   Product     10 non-null    str
2   Category    10 non-null    str
3   Region      10 non-null    str
4   Quantity    10 non-null    int64
5   Price       10 non-null    int64
6   Sales       10 non-null    int64
dtypes: int64(4), str(3)
memory usage: 910.0 bytes
None

Descriptive statistics:

```

	Order_ID	Quantity	Price	Sales
count	10.00000	10.00000	10.00000	10.00000
mean	105.50000	3.30000	24700.00000	60260.00000
std	3.02765	1.63639	18955.44483	42596.71870
min	101.00000	1.00000	1200.00000	5400.00000
25%	103.25000	2.00000	7500.00000	21250.00000
50%	105.50000	3.00000	25500.00000	62000.00000
75%	107.75000	4.75000	31500.00000	90750.00000
max	110.00000	6.00000	55000.00000	130000.00000

Explanation:

- Reading and inspecting data in Pandas helps us load a CSV file and understand its structure and contents.
- The program uses `pd.read_csv()` to load `sales_dataset.csv` into a DataFrame.
- Functions like `head()`, `tail()`, `shape`, `columns`, `dtypes`, and `info()` are used to inspect the dataset.
- `describe()` provides descriptive statistics such as count, mean, minimum, maximum, and standard deviation for numerical data.

Task 6: Selecting, Filtering & Sorting Data

program

```
import pandas as pd
```

```
df = pd.read_csv("sales_dataset.csv")
print("Selected columns:")
print(df[["Product", "Region", "Sales"]])
print("\nSelected rows (first 3):")
print(df.iloc[:3])
print("\nSales greater than 60000:")
print(df[df["Sales"] > 60000][["Product", "Region", "Sales"]])

print("\nMultiple conditions (South and Sales > 50000):")
print(df[(df["Region"] == "South") & (df["Sales"] > 50000)]
      [["Product", "Sales"]])

print("\nAscending order by Sales:")
print(df.sort_values("Sales")[["Product", "Sales"]])

print("\nDescending order by Sales:")
print(df.sort_values("Sales", ascending=False)[["Product", "Sales"]])
```

Output:

```
Selected columns:
  Product Region  Sales
0  Laptop  South 100000
1   Phone  North  75000
2 Headphones  South  10000
3   Tablet  East   60000
4  Laptop  West   55000
5   Phone  North  96000
6   Mouse  South   7200
7  Keyboard  West   5400
8   Tablet  East   64000
9   Phone  South 130000

Selected rows (first 3):
  Order_ID  Product  Category  Region  Quantity  Price  Sales
0       101   Laptop  Electronics  South         2   50000 100000
1       102    Phone  Electronics  North         3   25000  75000
2       103 Headphones  Accessories  South         5    2000  10000

Sales greater than 60000:
  Product Region  Sales
0  Laptop  South 100000
1   Phone  North  75000
5   Phone  North  96000
8  Tablet  East   64000
9   Phone  South 130000

Multiple conditions (South and Sales > 50000):
  Product  Sales
0  Laptop 100000
9   Phone 130000

Ascending order by Sales:
  Product  Sales
7  Keyboard   5400
6   Mouse   7200
2 Headphones 10000
4  Laptop   55000
3   Tablet  60000
8   Tablet  64000
1   Phone   75000
5   Phone  96000
0  Laptop 100000
9   Phone 130000

Descending order by Sales:
  Product  Sales
9   Phone 130000
0  Laptop 100000
5   Phone  96000
1   Phone  75000
8   Tablet  64000
3   Tablet  60000
4  Laptop   55000
2 Headphones 10000
6   Mouse   7200
7  Keyboard   5400
```

Explanation:

- Data selection, filtering, and sorting in Pandas are used to find and organize specific information from a dataset.
- The program selects specific columns and rows using `df[[]]` and `iloc`, and filters sales greater than 60000.
- It uses multiple conditions to find records from the South region where sales are greater than 50000.
- The `sort_values()` function arranges the sales data in ascending and descending order.

Task-7: Handling Missing Values

Program

```
import pandas as pd
df = pd.read_csv("sales_missing_dataset.csv")
print("Missing values before handling:")
print(df.isnull().sum())
print("\nRows after removing missing values:")
print(df.dropna())
cleaned = df.copy()
cleaned["Price"] = cleaned["Price"].fillna(cleaned["Price"].mean())
cleaned["Quantity"] = cleaned["Quantity"].fillna(cleaned["Quantity"].median())
cleaned["Region"] = cleaned["Region"].fillna("Unknown")
print("\nDataset after filling missing values:")
print(cleaned)
```

Output:

```
Missing values before handling:
Order_ID    0
Product     0
Category    0
Region      1
Quantity    1
Price       1
Sales       0
dtype: int64

Rows after removing missing values:
   Order_ID  Product  Category  Region  Quantity  Price  Sales
0        101  Laptop  Electronics  South      2.0  50000.0  100000
1        102   Phone  Electronics  North      3.0  25000.0   75000
3        104  Tablet  Electronics  East       2.0  30000.0   60000
4        105  Laptop  Electronics  West       1.0  55000.0   55000
6        107   Mouse  Accessories  South      6.0   1200.0    7200
7        108  Keyboard  Accessories  West      3.0   1800.0    5400
9        110   Phone  Electronics  South      5.0  26000.0  130000

Dataset after filling missing values:
   Order_ID  Product  Category  Region  Quantity  Price  Sales
0        101  Laptop  Electronics  South      2.0  50000.000000  100000
1        102   Phone  Electronics  North      3.0  25000.000000   75000
2        103  Headphones  Accessories  South      5.0  27222.222222   10000
3        104  Tablet  Electronics  East       2.0  30000.000000   60000
4        105  Laptop  Electronics  West       1.0  55000.000000   55000
5        106   Phone  Electronics  North      3.0  24000.000000   96000
6        107   Mouse  Accessories  South      6.0   1200.000000    7200
7        108  Keyboard  Accessories  West      3.0   1800.000000    5400
8        109  Tablet  Electronics  Unknown      2.0  32000.000000   64000
9        110   Phone  Electronics  South      5.0  26000.000000  130000
```

Note: Handling missing data is important because clean data improves the accuracy and reliability of analysis.

Explanation:

- Handling missing data in Pandas is used to identify and replace empty or missing values in a dataset.
- The program uses `isnull().sum()` to count missing values and `dropna()` to remove rows containing missing data.
- Missing Price values are replaced with the mean, while missing Quantity values are replaced with the median.
- Missing Region values are replaced with "Unknown", creating a cleaner and more complete dataset.

Task -8:Merge,Concatenate,Groupby &Pivot Table

Program

```
import pandas as pd

df1 = pd.read_csv("sales_part1.csv")
df2 = pd.read_csv("sales_part2.csv")

# Merge using the common Category column
category_sales = pd.DataFrame({
    "Category": ["Electronics", "Accessories"],
    "Target": [250000, 30000]
})

category_actual = (
    pd.concat([df1, df2])
    .groupby("Category", as_index=False)["Sales"]
    .sum()
    .rename(columns={"Sales": "Actual_Sales"})
)

merged = pd.merge(category_actual, category_sales, on="Category")
print("Merged DataFrame:")
print(merged)

concatenated = pd.concat([df1, df2], ignore_index=True)
print("\nConcatenated DataFrame:")
print(concatenated)

print("\nGroupBy - total sales by Region:")
print(concatenated.groupby("Region")["Sales"].sum())
```

```
print("\nGroupBy - average sales by Category:")
print(concatenated.groupby("Category")["Sales"].mean())
```

```
pivot = pd.pivot_table(
    concatenated,
    values="Sales",
    index="Region",
    columns="Category",
    aggfunc="sum",
    fill_value=0
)
```

```
print("\nPivot Table:")
print(pivot)
```

Output:

```
... Merged DataFrame:
   Category  Actual_Sales  Target
0  Accessories      22600   30000
1  Electronics     580000  250000

Concatenated DataFrame:
   Order_ID  Product  Category  Region  Quantity  Price  Sales
0        101   Laptop  Electronics  South         2   50000  100000
1        102    Phone  Electronics  North         3   25000   75000
2        103  Headphones  Accessories  South         5    2000   10000
3        104   Tablet  Electronics  East         2   30000   60000
4        105   Laptop  Electronics  West         1   55000   55000
5        106    Phone  Electronics  North         4   24000   96000
6        107    Mouse  Accessories  South         6    1200    7200
7        108  Keyboard  Accessories  West         3    1800    5400
8        109   Tablet  Electronics  East         2   32000   64000
9        110    Phone  Electronics  South         5   26000  130000

GroupBy - total sales by Region:
Region
East      124000
North     171000
South     247200
West       60400
Name: Sales, dtype: int64

GroupBy - average sales by Category:
Category
Accessories      7533.333333
Electronics     82857.142857
Name: Sales, dtype: float64

Pivot Table:
Category  Accessories  Electronics
Region
East           0       124000
North          0       171000
South        17200       230000
West          5400        55000
```


Explanation:

- Merging, concatenating, grouping, and pivoting in Pandas are used to combine, summarize, and analyze data from multiple datasets.
- The program combines two sales files using `concat()` and merges actual sales with target sales using `merge()` based on the common `Category` column.
- `groupby()` calculates total sales by `Region` and average sales by `Category`.
- `pivot_table()` creates a summary table showing total sales for each `Category` across different `Regions`.

Task-9: Exporting Data

Program

```
import pandas as pd
df = pd.read_csv("sales_dataset.csv")
df["Sales_per_Unit"] = df["Sales"] / df["Quantity"]
output_file = "processed_sales.csv"
df.to_csv(output_file, index=False)
check = pd.read_csv(output_file)
print("File exported successfully:", output_file)
print("Rows and columns:", check.shape)
print("\nExported data:")
print(check.head())
```

Output:

```
File exported successfully: processed_sales.csv
Rows and columns: (10, 8)

Exported data:
  Order_ID  Product  Category Region  Quantity  Price  Sales  \
0       101   Laptop  Electronics  South         2   50000  100000
1       102    Phone  Electronics  North         3   25000   75000
2       103  Headphones  Accessories  South         5    2000   10000
3       104   Tablet  Electronics  East         2   30000   60000
4       105   Laptop  Electronics  West         1   55000   55000

Sales_per_Unit
0       50000.0
1       25000.0
2        2000.0
3       30000.0
4       55000.0
```


Explanation:

- Exporting data in Pandas is used to save a processed DataFrame into a new CSV file for future use.
- The program calculates a new Sales_per_Unit column by dividing Sales by Quantity.
- The to_csv() function saves the updated DataFrame as processed_sales.csv without storing the DataFrame index.
- The exported file is then read using read_csv() and checked by displaying its shape and first five rows.

Task-10: Mini Data Analysis Project

Program

```
import pandas as pd
import numpy as np

df = pd.read_csv("sales_dataset.csv")

print("1. Dataset shape:", df.shape)

print("\n2. Total Sales:", df["Sales"].sum())
print("3. Average Sales:", df["Sales"].mean())

print("\n4. Sales by Region:")
print(df.groupby("Region")["Sales"].sum())

print("\n5. Sales by Category:")
print(df.groupby("Category")["Sales"].sum())

print("\n6. Top 3 products by sales:")
print(df.groupby("Product")["Sales"].sum().sort_values(ascending=False).head(3))

pivot = pd.pivot_table(
    df,
    values="Sales",
    index="Region",
    columns="Category",
    aggfunc="sum",
    fill_value=0
)
```

```

print("\n7. Pivot Table:")
print(pivot)

df["Sales_per_Unit"] = df["Sales"] / df["Quantity"]
df.to_csv("cleaned_sales_analysis.csv", index=False)

print("\n8. Cleaned dataset exported successfully.")

```

Output:

```

1. Dataset shape: (10, 7)

2. Total Sales: 602600
3. Average Sales: 60260.0

4. Sales by Region:
Region
East      124000
North     171000
South     247200
West       60400
Name: Sales, dtype: int64

5. Sales by Category:
Category
Accessories      22600
Electronics     580000
Name: Sales, dtype: int64

6. Top 3 products by sales:
Product
Phone      301000
Laptop     155000
Tablet     124000
Name: Sales, dtype: int64

7. Pivot table:
Category Accessories Electronics
Region
East              0         124000
North              0         171000
South           17200         230000
West             5400          55000

8. Cleaned dataset exported successfully.

```

Explanation:

- Sales Data Analysis using Pandas is a mini project used to read, process, analyze, and summarize sales data from a CSV file.
- The program calculates total sales, average sales, regional sales, category-wise sales, and top 3 products using Pandas functions.
- It uses `groupby()` and `pivot_table()` to organize and compare sales data based on Region and Category.
- Finally, it calculates `Sales_per_Unit` and exports the processed data to `cleaned_sales_analysis.csv` for future use.